



Headless CMS

Technical Documentation NL

Strapi Backend • SvelteKit Frontend

Responsible IT
Juni 2026



Inhoudsopgave

1. Inleiding & Projectoverzicht	4
Repositories.....	4
Hoe de twee projecten verbonden zijn.....	4
2. Systeemarchitectuur	5
Architectuurdiagram	5
Fallback / lokale ontwikkeling.....	5
3. Backend — Strapi CMS	6
Wat Strapi doet.....	6
Mappenstructuur	6
Inhoudstypen	7
Componenten	10
Geïnstalleerde plug-ins	10
Aangepast eindpunt: /api/rebuild.....	11
Configuratiebestanden	11
4. Frontend — SvelteKit.....	12
Wat SvelteKit doet.....	12
Mappenstructuur	12
Routes	13
Omgevingsvariabelen.....	14
Datafetchingpatroon	14
Vooraf renderen (entries()-functies).....	15
Svelte-componenten	15
De “revealed”-sneltoets	15
5. Aan de slag — Lokale installatie.....	16
Vereisten	16
1. Strapi backend instellen	16
2. SvelteKit frontend instellen.....	16
3. Vereiste Strapi-omgevingsvariabelen	17
4. Een Strapi API-token aanmaken.....	17
6. Deployment.....	18
Overzicht	18
Strapi backend deployment.....	18
SvelteKit frontend deployment	18
Automatische rebuild bij contentpublicatie.....	18
7. Handleiding voor contenteditors	19

Inloggen.....	19
Een nieuw studentproject aanmaken.....	19
Een nieuw onderzoeksproject aanmaken.....	19
De navigatie bewerken.....	19
Partners beheren.....	20
Generieke pagina's.....	20
Concept versus Gepubliceerd.....	20
8. Ontwikkelaarshandleiding.....	21
Een nieuw inhoudstype toevoegen aan Strapi.....	21
Een nieuwe route toevoegen aan de frontend.....	21
Een nieuw Svelte-component toevoegen.....	21
Werken met de Strapi REST API.....	21
Strapi TypeScript-types.....	22
Codestijl en tooling.....	22
9. Snelle API-referentie.....	23
Basis-URL.....	23
Authenticatie.....	23
Eindpunten.....	23
Voorbeeldreactiestructuur (pagina).....	24
10. Probleemoplossing.....	25
11. Herstel van het Strapi-schema.....	26
Het probleem begrijpen.....	26
Wat gebeurt er tijdens een rebuild?.....	27
Symptomen.....	27
Oorzaak.....	27
Checklist voor herstel.....	28
12. Woordenlijst.....	29
13. Huidige status.....	30
Huidig process.....	30
Overgebleven taken.....	30

1. Inleiding & Projectoverzicht

Dit document beschrijft het **Headless CMS** gebouwd door responsibleIT. Het bestaat uit twee afzonderlijke repositories die samenwerken: een **Strapi backend** die content beheert en beschikbaar stelt via een REST API, en een **SvelteKit frontend** die de content ophaalt tijdens het bouwen om een volledig statische website te genereren.

De architectuur is “headless” omdat het CMS (Strapi) geen ingebouwde website heeft — het levert alleen data via een API. De visuele website wordt apart gebouwd en beheerd in SvelteKit, die de API consumeert.

Repositories

Backend — Strapi CMS	
Repo	responsibleIT/test-website-strapi
Branch	main
Technologie	Strapi v5 (Node.js / TypeScript)
Standaardpoort	1337
Database	SQLite (ontwikkeling) / PostgreSQL of MySQL (productie)
Doel	Contentbeheer, REST API, media-uploads, deploy webhook

Frontend — SvelteKit	
Repo	responsibleIT/headless-test-website-strapi
Branch	svelte-rewrite
Technologie	SvelteKit + Svelte 5 (static adapter)
Uitvoer	Volledig vooraf gegenereerde statische HTML/CSS/JS
Doel	Strapi-data ophalen tijdens het bouwen en publieke website renderen

Hoe de twee projecten verbonden zijn

Tijdens de build roept SvelteKit de Strapi REST API aan om alle content op te halen. Het genereert één HTML-bestand per pagina en levert een statische map op die naar elke webhost kan worden gedeployed (Wij gebruiken Coolify). Tijdens de Wanneer een bezoeker de site laadt is er geen verbinding meer met Strapi de bezoeker ontvangt alleen de vooraf gebouwde statische bestanden.

Deploy bij publicatie

Wanneer een contenteditor content publiceert of bijwerkt in Strapi, verstuurt Strapi een deploy webhook naar Coolify die een nieuwe build van de SvelteKit-frontend triggert, waardoor de wijzigingen zichtbaar worden op de live site.

2. Systeemarchitectuur

Architectuurdiagram

Flow

Contenteditor → Strapi Beheerpaneel → Strapi REST API → SvelteKit build → Statische bestanden → Bezoekerbrowser

Laag	Wat het doet	Technologie
Content Type builder	Visuele definitie van contenttypen en componenten.	Strapi v5 Admin UI
Content management	Editors maken/bewerken/publiceren content	SQLite / PostgreSQL
Data storage	Stelt content beschikbaar als JSON via REST	Strapi REST API (port 1337)
API	Haalt API op en genereert statische HTML	SvelteKit + adapter-static
Build	Serveert statische bestanden aan bezoekers	Coolify / elke statische host
Hosting	Start een nieuwe build bij contentwijziging	Coolify deploy webhook

Fallback / lokale ontwikkeling

De frontend ondersteunt twee API-eindpunten: een primaire externe Strapi (`PUBLIC_STRAPI_API_URL`) en een lokale fallback (`PUBLIC_LOCAL_API_URL`). Als de externe Strapi niet bereikbaar is (bijv. tijdens lokale ontwikkeling), probeert de build automatisch de lokale instantie. Hierdoor kunnen ontwikkelaars offline werken met een lokale Strapi zonder code aan te passen.

3. Backend — Strapi CMS

Wat Strapi doet

Strapi is het “brein” van het contentsysteem. Het biedt:

- Een visueel beheerpaneel op /admin waar editors alle content beheren
- Een REST API op /api/* die de frontend bevraagt tijdens het bouwen
- Mediabeheer — bestandsuploads opgeslagen in /public/uploads/
- Gebruikers- & rolbeheer — bepaalt wie de API mag benaderen en met welke rechten
- Concept- & publicatieworkflow — content kan als concept worden opgeslagen vóór publicatie
- Een deploy webhook-eindpunt dat een frontend-rebuild triggert in Coolify (raket)

Mappenstructuur

```

strapi-backend/
├── config/                ← Server-, database-, plugin- en middleware-configuratie
│   ├── database.ts       ← DB-verbinding (SQLite dev, Postgres/MySQL prod)
│   ├── server.ts         ← Host- en poortinstellingen
│   ├── plugins.ts        ← Plug-ininstellingen (code editor, uploads)
│   └── middlewares.ts    ← CORS, beveiligingsheaders, logging
├── src/
│   ├── api/              ← Één map per inhoudstype
│   │   ├── page/         ← Generieke CMS-pagina
│   │   ├── navigation/   ← Sitenavigatie (single type)
│   │   ├── partner/      ← Partnerorganisaties
│   │   ├── research-detail-page/ ← Onderzoeksprojectinvoeren
│   │   ├── student-detail-page/ ← Studentprojectinvoeren
│   │   └── rebuild/      ← Aangepast deploy webhook-eindpunt
│   ├── components/       ← Herbruikbare veldgroepen, gebruikt binnen inhoudstypen
│   │   ├── component/    ← Bronnen, Buttons, Onderwijseenheid, ResearcherData
│   │   ├── layout/       ← Section (pagina-lay-outblokken)
│   │   └── navigation/   ← NavItems
│   └── index.ts          ← Strapi lifecycle hooks (register / bootstrap)
├── public/uploads/       ← Mediabestanden geupload via het beheerpaneel
└── types/generated/      ← Gegeneerde TypeScript-types (niet bewerken)

```

Inhoudstypen

Een inhoudstype is een gestructureerd datamodel — zoals een databasetabel — dat editors invullen via het beheerpaneel.

Pagina (collectietype)

Generieke websitepagina's voor de homepage en andere platte contentpagina's.

Veld	Type	Beschrijving
title	Tekst (verplicht)	De weergavenaam van de pagina
slug	UID → titel	URL-veilige identifier, automatisch gegenereerd vanuit de titel (klik op herladen wanneer je het aanpast)
metaBeschrijving	Tekst	SEO-beschrijving die in zoekresultaten wordt weergegeven
video	Media (bestand/video)	Optionele videobijlage
sections	Component (Section, repeatable)	Lijst van lay-outsecties op de pagina

Navigatie (enkel type)

Er is slechts één navigatierecord — het definieert het sitebreed navigatiemenu.

Veld	Type	Beschrijving
navItems	Component (navItems, repeatable)	Geordende lijst van navigatielinks (label, url, volgorde)

Partner (collectietype)

Organisaties die samenwerken aan student- of onderzoeksprojecten.

Veld	Type	Beschrijving
name	Tekst (verplicht)	Naam van de partnerorganisatie
student_detail_page	Relatie (veel-op-veel)	Studentprojecten waaraan deze partner gekoppeld is
research_detail_pages	Relatie (veel-op-veel)	Onderzoeksprojecten waaraan deze partner gekoppeld is

Studentdetailpagina (collectietype)

Één invoer per studentproject. Deze verschijnen onder /student-projecten/ op de website.

Veld	Type	Opmerkingen
ProjectTitel	Tekst (verplicht)	Projecttitel, minimaal 2 tekens
Slug	UID → ProjectTitel	Automatische URL-slug, gebruikt in de pagina-URL (klik op herladen wanneer je het aanpast)
ProjectBeschrijving	Tekst	Korte beschrijving
LesJaar	Geheel getal (0–99)	Academisch jaarnummer, standaard 26
AantalStudenten	Geheel getal (min. 1)	Aantal studenten in het project
Opleiding	Meervoudige keuze	Opleiding(en): HBO-ICT, CMD, CO, CB, FDND
SchoolJaar	Meervoudige keuze	Studiejaar: 1, 2, 3 of 4
Niveau	Meervoudige keuze (max. 1)	Niveau: Bachelor, Master of AD
Doorlooptijd	Tekst	Projectduur (vrije tekst, bijv. “8 weken”)
VideoEmbed	Tekst	Embed-URL voor een video (bijv. YouTube iframe src)
Foto	Media (afbeelding/bestand)	Projectfoto
Audio	Media (meerdere, audio)	Één of meer audiobestanden
Onderwijseenheid	Component	Onderwijseenheid met label en URL
Bronnen	Component (repeatable)	Bron-/referentie-URL's
Partners	Relatie (veel-op-veel)	Partnerorganisaties
P5	Code-editor (JS)	Optionele P5.js-schets insluitcode

Onderzoeksdetailpagina (collectietype)

Één invoer per onderzoeksproject. Deze verschijnen onder /research-projecten/ op de website.

Veld	Type	Opmerkingen
ProjectTitel	Tekst (verplicht, uniek)	Titel van het onderzoeksproject
Slug	UID → ProjectTitel	Automatische URL-slug (klik op herladen wanneer je het aanpast)
ProjectBeschrijving	Tekst	Projectbeschrijving
Jaar	Geheel getal	Jaar waarin het onderzoek is uitgevoerd
Duur	Geheel getal	Duur (in een eenheid — bijv. maanden)
Regeling	Tekst	Financieringsregeling of programma
VideoEmbed	Tekst	Embed-URL voor een video
Foto	Media (afbeelding/bestand)	Projectfoto
Audio	Media (audio/bestand)	Audiobestand
ResearcherData	Component (repeatable)	Naam + e-mailparen van onderzoekers
Bronnen	Component (repeatable)	Bron-/referentie-URL's
partners	Relatie (veel-op-veel)	Partnerorganisaties
research_pages	Zelfrelatie (veel-op-veel)	Gerelateerde onderzoeksprojecten
P5	Code-editor (JS)	Optionele P5.js-schets insluitcode

Componenten

Componenten zijn herbruikbare veldgroepen. In tegenstelling tot inhoudstypen hebben ze geen eigen API-eindpunt — ze bestaan alleen binnen een bovenliggend inhoudstype.

Component	Velden	Gebruikt in
Section (layout)	type (hero/text/cta/gallery), heading, body, image, buttons[]	Page
Buttons (component)	label, url, style (primary/secondary/outline), openInNewTab	Section
navItems (navigation)	label, url, order	Navigation
Bronnen (component)	Bron (unieke string-URL)	Student- + Research-pagina's
Onderwijseenheid (component)	label, URL	Studentdetailpagina
ReasearcherData (component)	Naam, Email	Onderzoeksdetailpagina

Geïnstalleerde plug-ins

Plug-in	Doel
strapi-plugin-multi-select	Voegt een meervoudig keuzeveld toe (gebruikt voor Opleiding, SchoolJaar, Niveau)
strapi-code-editor-custom-field	Voegt een code-editorveld toe in het beheerpaneel (gebruikt voor P5-schetsen)
strapi-plugin-config-sync	Synchroniseert plug-inconfiguratie naar/van JSON-bestanden in versiebeheer
@strapi/plugin-users-permissions	Beheert publieke/geverifieerde API-toegangsrechten
@strapi/plugin-cloud	Optionele ondersteuning voor Strapi Cloud-deployment

Aangepast eindpunt: `/api/rebuild`

De map `src/api/rebuild/` bevat een aangepast POST-eindpunt. Wanneer dit wordt aangeroepen, leest het de omgevingsvariabele `COOLIFY_DEPLOY_WEBHOOK` en doet het een GET-aanroep naar de deploy webhook-URL van Coolify om een nieuwe build van de SvelteKit-frontend te triggeren.

Dit eindpunt heeft geen authenticatie (`auth: false`), dus het is ontworpen om door Strapi's ingebouwde webhooksysteem te worden aangeroepen na een content-publicatiegebeurtenis. Het token wordt uit de webhook-URL zelf geëxtraheerd en als Bearer-header verstuurd.

Configuratiebestanden

config/database.ts

Ondersteunt SQLite (standaard voor ontwikkeling), PostgreSQL en MySQL. De actieve database wordt geselecteerd via de omgevingsvariabele `DATABASE_CLIENT`. SQLite slaat zijn bestand op in `.tmp/data.db`.

config/server.ts

Bindt standaard aan `0.0.0.0` op poort 1337. Te overschrijven via de omgevingsvariabelen `HOST` en `PORT`.

config/middlewares.ts

Schakelt de standaard Strapi middleware-stack in. Voegt een aangepast Content Security Policy toe om Monaco-editorscripts van `cdn.jsdelivr.net` toe te staan (vereist voor de code-editorplug-in).

config/plugins.ts

Schakelt `strapi-code-editor-custom-field` in en configureert afbeeldingsbreekpunten voor de upload-plug-in (`xsmall: 64px → xlarge: 1920px`).

4. Frontend — SvelteKit

Wat SvelteKit doet

SvelteKit is het framework dat Strapi's API-data omzet in een publieke website. Belangrijkste kenmerken:

- **Statische sitegeneratie (SSG):** elke pagina wordt vooraf gegenereerd naar gewone HTML tijdens het bouwen. Er draait geen servercode wanneer een bezoeker de site opent.
- **Svelte 5 runes-modus:** het project gebruikt Svelte's nieuwste reactiviteitsmodel (`$props()`, `$state()`, `$derived()`) door de hele codebase.
- **adapter-static:** de builduitvoer is een map met HTML/CSS/JS-bestanden die naar elke statische host kunnen worden gedeployed.
- **Fallback 404.html:** de adapter is geconfigureerd met een fallbackpagina voor client-side routing op hosts die dit ondersteunen.

Mappenstructuur

svelte-frontend/src/	
├─ app.html	← HTML shell (head, body, favicon, reveal script)
├─ lib/	
│ └─ assets/	← favicon.svg
│ └─ components/	
│ └─ PageCard/	← Uitklapbare kaart met Page + secties
│ └─ SectionCard/	← Rendert één Section-componentblok
│ └─ index.js	← (leeg barrel-bestand)
├─ routes/	
│ └─ +layout.js	← Stelt prerender=true globaal in
│ └─ +layout.svelte	← Root layout: CSS, favicon, sneltoets
│ └─ +page.server.js	← Haalt homepagina + paginalijst op
│ └─ +page.svelte	← Rendert homepagina met secties en paginalijst
│ └─ [slug]/	← Dynamische route voor generieke CMS-pagina's
│ └─ +page.server.js	← Haalt pagina op via slug + prerender-entries
│ └─ +page.svelte	← Rendert één pagina
│ └─ research-projecten/	
│ └─ +page.server.js	← Haalt alle onderzoeksprojecten op
│ └─ +page.svelte	← Toont alle onderzoeksprojecten
│ └─ [slug]/	← Onderzoeksprojectdetailpagina
│ └─ student-projecten/	
│ └─ +page.server.js	← Haalt alle studentprojecten op
│ └─ +page.svelte	← Toont alle studentprojecten
│ └─ [slug]/	← Studentprojectdetailpagina
└─ styles/	
│ └─ general.css	← Globale reset, bodyfont, lay-out-utilities
│ └─ pages/home.css	← Decoratieve stijlen specifiek voor de homepagina

Routes

URL-patroon	Contentbron	Opmerkingen
/	Page met slug “home” + alle pagina's	Landingspagina met sectierenderer
/[slug]	Page-collectie (op slug)	Elke generieke CMS-pagina; gereserveerde slugs uitgesloten
/student-projecten	student-detail-pages-collectie	Toont alle studentprojecten
/student-projecten/[slug]	student-detail-pages (op Slug)	Volledige studentprojectdetails
/research-projecten	research-detail-pages-collectie	Toont alle onderzoeksprojecten
/research-projecten/[slug]	research-detail-pages (op Slug)	Volledige onderzoeksprojectdetails

De slugs student-projecten, research-projecten en home zijn “gereserveerd” — de catch-all route [slug] slaat deze expliciet over zodat ze worden afgehandeld door hun eigen toegewijde routemappen.

Omgevingsvariabelen

Maak een .env-bestand aan in de root van svelte-frontend met deze variabelen:

```
# Externe Strapi (eerst gebruikt)
PUBLIC_STRAPI_API_URL='http://gwtti5qjvak0lg9nots6p1uk.185.88.141.173.sslip.io'

LOCAL_API_TOKEN=ce8d69900f72bc4890ebd686038f347fa9c7414fea6a00e9721524174c1d588f66c639d2
9fb82ddc32e5530d02068762dff14794c22524af569dd97ed7b9ff25fa901e672ea0611b45c9227acc6e6389
3ff3241bca3d36e9c500dc84c4e9a622a122cb4e81ea9d4fbb7fef63451f8c68e808303e246c55a52670d4c0
5caeaebc

# Lokale Strapi-fallback (gebruikt als externe niet beschikbaar is)
PUBLIC_LOCAL_API_URL=http://localhost:1337
LOCAL_API_TOKEN=your_local_strapi_api_token_here
```

Variabele	Bereik	Beschrijving
PUBLIC_STRAPI_API_URL	Publiek (zichtbaar voor browser)	Basis-URL van de primaire Strapi-instantie. Ook gebruikt als prefix voor media-URL's.
API_TOKEN	Privé (alleen buildserver)	Strapi API-token voor de externe instantie.
PUBLIC_LOCAL_API_URL	Publiek	Basis-URL van de lokale/fallback Strapi-instantie.
LOCAL_API_TOKEN	Privé	Strapi API-token voor de lokale instantie.

Belangrijk

PUBLIC_-variabelen zijn ingebed in de gebouwde JavaScript en zichtbaar voor eindgebruikers. Zet nooit geheimen in PUBLIC_-variabelen. De API_TOKEN-variabelen zijn privé (alleen gebruikt tijdens het bouwen door +page.server.js-bestanden) en worden niet in de uitvoer opgenomen.

Datafetchingpatroon

Elk +page.server.js-bestand volgt hetzelfde fetchpatroon:

- **Primair:** Probeer de externe Strapi-URL met API_TOKEN.
- **Fallback:** Als het primaire mislukt, probeer de lokale Strapi-URL met LOCAL_API_TOKEN.
- **Graceful degradation:** Geef null of een lege array terug als beide mislukken, zodat de pagina een zinvolle lege toestand kan weergeven in plaats van de build te laten crashen.

Vooraf renderen (entries()-functies)

Voor dynamische routes zoals `[slug]` moet SvelteKit vooraf weten welke URL's bestaan om ze te kunnen pre-renderen. Elke dynamische route exporteert een `entries()`-functie die de lijst van alle records uit Strapi ophaalt en een array van slug-objecten teruggeeft. SvelteKit pre-renderd vervolgens één HTML-bestand per slug.

```
// Voorbeeld uit [slug]/+page.server.js
export async function entries() {
  const json = await fetchWithFallback('/api/pages', { 'pagination[pageSize]': '100' });
  return json.data
    .filter(p => p.slug && !RESERVED_SLUGS.has(p.slug))
    .map(p => ({ slug: p.slug }));
}
```

Svelte-componenten

SectionCard

Rendert één `layout.section`-componentblok. Toont de sectietype-badge, een optionele afbeelding (opgehaald uit Strapi via de volledige basis-URL), en een key-value lijst van alle niet-genegeerde velden. Negeert `id`, `__component`, `type`, `image` en `buttons` (deze worden apart gerenderd).

PageCard

Een uitklapbaar kaartcomponent dat op de homepagina wordt getoond. Toont de paginatitel (als link), metabeschrijving, sectie-aantalbadge, videobadge en publicatiestatus. Door op de kaart te klikken wordt deze uitgeklapt en worden alle `SectionCard`-kindelementen getoond. Wordt gebruikt als overzicht voor ontwikkelaars.

De “revealed”-sneltoets

De layout bevat een verborgen toetsenbordsneltoets: door achtereenvolgens Enter, Enter, Backspace te drukken wordt een `.revealed-class` toegevoegd aan `<body>`. Dit onthult decoratieve `body::before` / `body::after` pseudo-elementen die gedefinieerd zijn in `home.css`. De `revealed-status` blijft 3 dagen bewaard in `localStorage`. Dit is bewust niet gedocumenteerd in de UI — het is een preview-functie voor ontwikkelaars/klanten.

5. Aan de slag — Lokale installatie

Vereisten

- Node.js 20 of hoger
- pnpm (of npm / yarn)
- Git

1. Strapi backend instellen

```
# Clone de backend
git clone https://github.com/responsibleIT/test-website-strapi.git
cd test-website-strapi

# Installeer afhankelijkheden
pnpm install

# Kopieer het omgevingsbestand en vul de waarden in
cp .env.example .env # (of maak .env handmatig aan – zie sectie 5.3)

# Start in ontwikkelingsmodus (met hot reload)
pnpm dev

# Strapi draait nu op http://localhost:1337
# Beheerpaneel: http://localhost:1337/admin
```

2. SvelteKit frontend instellen

```
# Clone de frontend (svelte-rewrite branch)
git clone -b svelte-rewrite https://github.com/responsibleIT/headless-test-website-strapi.git
cd headless-test-website-strapi

# Installeer afhankelijkheden
pnpm install

# Maak het omgevingsbestand aan
cat > .env << EOF
PUBLIC_STRAPI_API_URL=https://your-remote-strapi.example.com
API_TOKEN=your_remote_token
PUBLIC_LOCAL_API_URL=http://localhost:1337
LOCAL_API_TOKEN=your_local_token
EOF

# Start de ontwikkelingsserver (voert build-time fetches uit bij elke paginalaad)
pnpm dev

# Site is beschikbaar op http://localhost:5173
```

3. Vereiste Strapi-omgevingsvariabelen

Maak een .env-bestand aan in de strapi-backend-map:

```
HOST=0.0.0.0

PORT=1337

# Secrets

APP_KEYS=pQRX68rofI5LbnNh0xyDww==,4aJug/yBQ1wacVKd/qRc3A==,5kGNxLeYkxT04mBVZSuatQ==,e7ng
02tL6yitGeRH9ciQiw==

API_TOKEN_SALT=MrQVoexZGRNbJ+drRWF1Uw==

ADMIN_JWT_SECRET=r00YTsaE8yri1ulskfekvQ==

TRANSFER_TOKEN_SALT=jjGZ7iz2yfo9Ijyw39bSQQ==

ENCRYPTION_KEY=uwGW0YjbEckHPGwldYXewA==

# Database

DATABASE_CLIENT=sqlite

DATABASE_HOST=

DATABASE_PORT=

DATABASE_NAME=

DATABASE_USERNAME=

DATABASE_PASSWORD=

DATABASE_SSL=false

DATABASE_FILENAME=.tmp/data.db

JWT_SECRET=i0Eue1vCShkjp1NTgQJ3Lw==
```

4. Een Strapi API-token aanmaken

- Ga naar <http://localhost:1337/admin> → Instellingen → API-tokens
- Klik op “Nieuw API-token aanmaken”
- Geef het een naam, stel het type in op “Alleen lezen” (of “Volledige toegang” voor lokale ontwikkeling)
- Kopieer het token en zet het in de .env van de frontend als API_TOKEN of LOCAL_API_TOKEN
- Schakel in Instellingen → Gebruikers & Rechten → Rollen → Publiek “find” en “findOne” in voor elk inhoudstype dat de frontend nodig heeft

6. Deployment

Overzicht

Beide projecten worden gedeployed naar Coolify (een zelf-gehoste PaaS). De Strapi-backend draait als een persistente Node.js-server; de SvelteKit-frontend wordt gebouwd als een statische site en geserveerd als bestanden.

Strapi backend deployment

- Push de strapi-backend repo naar je Git remote.
- Maak in Coolify een nieuwe Applicatie aan die naar de repo verwijst.
- Stel het buildcommando in op `pnpm build` en het startcommando op `pnpm start`.
- Voeg alle vereiste omgevingsvariabelen toe (zie sectie 5.3).
- Stel voor productie `DATABASE_CLIENT` in op `postgres` of `mysql` en configureer de verbinding.
- Deploy. Strapi is toegankelijk op je gekozen domein op poort 1337 (of achter een reverse proxy op poort 443).

SvelteKit frontend deployment

- Push de headless-test-website-strapi repo (svelte-rewrite branch) naar je Git remote.
- Maak in Coolify een Statische Site Applicatie aan die naar de repo verwijst.
- Stel het buildcommando in op `pnpm build`.
- Stel de publicatiemap in op `build`.
- Voeg de omgevingsvariabelen toe (`PUBLIC_STRAPI_API_URL`, `API_TOKEN`, `PUBLIC_LOCAL_API_URL`, `LOCAL_API_TOKEN`).
- Deploy. Na een succesvolle build is de site live.

Automatische rebuild bij contentpublicatie

Wanneer een contenteditor nieuwe content publiceert in Strapi, moet de statische site opnieuw worden gebouwd. Dit wordt automatisch afgehandeld:

- **Strapi webhook:** Ga in het Strapi-beheerpaneel naar Instellingen → Webhooks → Nieuwe webhook aanmaken.
- **Gebeurtenis:** Selecteer “Entry → Publiceren”.
- **URL:** Wijs deze naar je Strapi API: `https://your-strapi.example.com/api/rebuild`
- **Coolify webhook:** De rebuild-controller leest `COOLIFY_DEPLOY_WEBHOOK` en roept Coolify's deploy API aan, die een nieuwe SvelteKit-build start.
- **Resultaat:** Binnen enkele minuten weerspiegelt de live site de nieuwe content.

7. Handleiding voor contenteditors

Inloggen

Ga naar de URL van je Strapi-beheerpaneel (bijv. <https://cms.jouwsite.nl/admin>). Log in met je inloggegevens. Heb je nog geen account, vraag dan een ontwikkelaar of beheerder om er een aan te maken onder Instellingen → Gebruikers.

Een nieuw studentproject aanmaken

- Klik in de linker zijbalk op Contentbeheer → Studentdetailpagina.
- Klik op “Nieuwe invoer aanmaken” (rechtsboven).
- Vul ProjectTitel in (verplicht). De Slug wordt automatisch gegenereerd.
- Vul ProjectBeschrijving, LesJaar, AantalStudenten in.
- Selecteer Opleiding, SchoolJaar (1–4), Niveau (Bachelor/Master/AD).
- Voeg optioneel Doorlooptijd, een Foto-afbeelding, Audiobestanden en een VideoEmbed-URL toe.
- Voeg Partners toe door op “Relatie toevoegen” te klikken en bestaande Partnerrecords te zoeken.
- Voeg Bronnen (bron-URL's) toe door op “+ Component toevoegen” te klikken.
- Klik als je klaar bent op “Publiceren” (niet alleen Opslaan). Publiceren triggert de website-rebuild.

Een nieuw onderzoeksproject aanmaken

Zelfde proces als studentprojecten, maar in Contentbeheer → Onderzoeksdetailpagina. Belangrijkste verschillen:

- Gebruik ReasearcherData om naam en e-mailparen van onderzoekers toe te voegen.
- Jaar is het jaar van het onderzoek (geheel getal).
- research_pages laat je gerelateerde onderzoeksprojecten aan elkaar koppelen.

De navigatie bewerken

- Ga naar Contentbeheer → Navigatie (dit is een single type — er is er maar één).
- Voeg navItems toe, verwijder ze of sorteer ze opnieuw. Elk heeft een label, url en volgordenummer.
- Publiceer als je klaar bent.

Partners beheren

- Ga naar Contentbeheer → Partners.
- Maak partnerinvoeren aan met een naam.
- Partners worden gekoppeld aan student-/onderzoekspagina's vanuit de formulieren van die pagina's.

Generieke pagina's

Gebruik Contentbeheer → Pagina voor elke andere pagina op de site (homepagina, over ons, contact, enz.). De homepagina moet slug = "home" hebben. Elke pagina kan meerdere secties hebben:

- **hero:** Grote introsectie met koptekst, inhoud, afbeelding, knoppen
- **text:** Bodytekst met optionele koptekst en afbeelding
- **cta:** Call to action met knoppen
- **gallery:** Afbeeldingsgaleriesectie

Concept versus Gepubliceerd

Alle inhoudstypen hebben **concept- en publicatie**ondersteuning. "Opslaan" slaat alleen een concept op — het verschijnt pas op de live website als je op "Publiceren" klikt. Je kunt concepten meerdere keren veilig bewerken en opslaan vóór publicatie.

8. Ontwikkelaarshandleiding

Een nieuw inhoudstype toevoegen aan Strapi

- Ga in het Strapi-beheerpaneel naar Content-Type Builder → Nieuw collectietype aanmaken.
- Definieer de velden. Let op: UID-velden genereren automatisch slugs van een doelveld.
- Sla op. Strapi herstart en genereert de standaardbestanden onder `src/api/`.
- Ga naar Instellingen → Rollen → Publiek, schakel `find` en `findOne` in voor het nieuwe type.
- Het nieuwe type krijgt automatisch een REST-eindpunt op `/api/<pluralName>`.

Een nieuwe route toevoegen aan de frontend

```
# Voorbeeld: een /nieuws route toevoegen
mkdir -p src/routes/nieuws
touch src/routes/nieuws/+page.server.js
touch src/routes/nieuws/+page.svelte
```

Exporteer in `+page.server.js` een `load()`-functie die data ophaalt uit Strapi en teruggeeft. Destructureer in `+page.svelte` `let { data } = $props()` en render de data. Exporteer voor dynamische slugs een `entries()`-functie die alle mogelijke slugs teruggeeft.

Een nieuw Svelte-component toevoegen

```
# Maak een componentmap aan
mkdir -p src/lib/components/MyComponent
touch src/lib/components/MyComponent/MyComponent.svelte
touch src/lib/components/MyComponent/MyComponent.module.css

# Importeer in een pagina
import MyComponent from '$lib/components/MyComponent/MyComponent.svelte';
```

Werken met de Strapi REST API

Alle eindpunten volgen het patroon `GET /api/<pluralName>`. Belangrijke queryparameters:

Parameter	Voorbeeld	Effect
filters	<code>filters[slug][\$eq]=home</code>	Filteren op veldwaarde
populate	<code>populate=*</code>	Alle relaties en componenten opnemen
populate (nested)	<code>populate[sections][populate]=*</code>	Geneste componenten diep populieren
pagination	<code>pagination[pageSize]=100</code>	Tot 100 records tegelijk ophalen
publicationState	<code>publicationState=live</code>	Alleen gepubliceerde records teruggeven (standaard)

Alle API-verzoeken moeten de header `Authorization: Bearer <token>` bevatten, waarbij het token wordt aangemaakt in Instellingen → API-tokens.

Strapi TypeScript-types

De map `types/generated/` bevat automatisch gegenereerde TypeScript-interfaces voor alle inhoudstypen. Deze worden automatisch opnieuw gegenereerd wanneer je `pnpm dev` of `pnpm build` uitvoert. Bewerk deze bestanden niet handmatig.

Codestijl en tooling

Tool	Configuratie	Uitvoeren met
Prettier	<code>.prettierrc</code> (frontend)	<code>pnpm format</code>
svelte-check	<code>jsconfig.json</code>	<code>pnpm check</code>
TypeScript	<code>tsconfig.json</code> (backend)	Gecontroleerd tijdens build

9. Snelle API-referentie

Basis-URL

`https://your-strapi.example.com`

Authenticatie

`Authorization: Bearer <API_TOKEN>`

Eindpunten

Methode + Pad	Beschrijving
GET /api/pages	Alle generieke pagina's weergeven
GET /api/pages?filters[slug][\$eq]=home&populate[sections][populate]=*	Homepagina met secties ophalen
GET /api/navigation?populate=*	Sitenavigatie ophalen
GET /api/partners	Alle partners weergeven
GET /api/student-detail-pages	Alle studentprojecten weergeven
GET /api/student-detail-pages?filters[Slug][\$eq]=my-slug&populate=*	Één studentproject ophalen op basis van slug
GET /api/research-detail-pages	Alle onderzoeksprojecten weergeven
GET /api/research-detail-pages?filters[Slug][\$eq]=my-slug&populate=*	Één onderzoeksproject ophalen op basis van slug
POST /api/rebuild	Coolify-rebuild triggeren (geen authenticatie vereist)

Voorbeeldreactiestructuur (pagina)

```
{
  "data": [
    {
      "id": 1,
      "documentId": "abc123...",
      "title": "Home",
      "slug": "home",
      "metaBeschrijving": "Welkom op onze site",
      "video": null,
      "sections": [
        {
          "id": 1,
          "__component": "layout.section",
          "type": "hero",
          "heading": "Welkom",
          "body": "Inleidende tekst.",
          "image": {
            "url": "/uploads/hero.jpg", "width": 1920, "height": 1080
          },
          "buttons": [
            { "label": "Lees meer", "url": "/about", "style": "primary" }
          ]
        }
      ],
      "publishedAt": "2025-01-01T00:00:00.000Z"
    }
  ],
  "meta": { "pagination": { "page": 1, "pageSize": 25, "total": 1 } }
}
```

10. Probleemoplossing

Probleem	Waarschijnlijke oorzaak	Oplossing
Frontend-build mislukt met "HTTP 401"	API-token ontbreekt of is onjuist	Controleer de .env API_TOKEN-waarde en zorg dat deze als omgevingsvariabele is ingesteld in Coolify
Pagina's worden niet weergegeven op de site na publicatie	Rebuild niet getriggerd	Controleer de Strapi webhook-configuratie; roep POST /api/rebuild handmatig aan; controleer de Coolify deploy-logs
"Geen homepagina gevonden in Strapi"	Er bestaat geen Pagina met slug "home"	Maak een Pagina-invoer aan in Strapi met de slug "home" en publiceer deze
Media-afbeeldingen worden niet geladen	Verkeerd basis-URL-prefix	Controleer of PUBLIC_STRAPI_API_URL exact overeenkomt met je Strapi-domein (geen afsluitende slash)
Strapi toont een leeg beheerpaneel na deployment	Beheerpaneel niet gebouwd	Voer pnpm build uit in Strapi of stel het buildcommando in in Coolify
SQLite werkt lokaal maar mislukt in productie	SQLite is niet geschikt voor productie	Stel DATABASE_CLIENT=postgres in voor productie en configureer de Postgres-omgevingsvariabelen
Svelte-typefouten bij controle	Ontbrekende svelte-kit-synchronisatie	Voer eerst pnpm prepare uit, dan pnpm check
Nieuw inhoudstype niet zichtbaar in API	Ontbrekende publieke rechten	Strapi-beheerpaneel → Instellingen → Rollen → Publiek → schakel find/findOne in voor het nieuwe type

11. Herstel van het Strapi-schema

Het probleem begrijpen

Strapi slaat informatie op twee plaatsen op:

1. Database

De database bevat onder andere:

- Artikelen
- Pagina's
- Producten
- Door gebruikers gegenereerde content
- Veldwaarden

Voorbeelden:

- Titel van een artikel
- Inhoud van een artikel
- SEO-beschrijving
- Ondertitel

Deze gegevens worden doorgaans opgeslagen in:

- PostgreSQL
- MySQL
- MariaDB

2. Schemabestanden

Strapi slaat de definities van contenttypes op als bestanden binnen het project.

Voorbeelden:

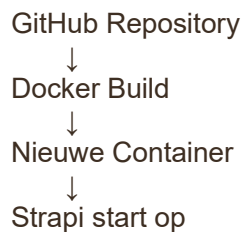
- `src/api/article/content-types/article/schema.json`
- `src/api/page/content-types/page/schema.json`
- `src/components/shared/seo.json`

Deze bestanden definiëren:

- Veldnamen
- Veldtypen
- Componenten
- Relaties
- Dynamic Zones

Wat gebeurt er tijdens een rebuild?

Wanneer Coolify een applicatie opnieuw bouwt, verloopt het proces als volgt:



Tijdens het opstarten laadt Strapi de schemadefinities vanuit de Git-repository.

Als de nieuwste schemabestanden niet in Git zijn opgeslagen:

Productiecontainer:

veld "subtitle" bestaat

GitHub:

veld "subtitle" ontbreekt

Na de deployment:

Strapi start zonder het veld "subtitle".

De gegevens blijven wel in de database aanwezig, maar Strapi weet niet langer hoe deze moeten worden weergegeven of bewerkt.

Symptomen

Je kunt de volgende problemen tegenkomen:

- Velden verdwijnen uit de Content Manager.
- Componenten verdwijnen.
- Relaties werken niet meer.
- Bestaande items kunnen niet meer worden bewerkt.
- Dynamic Zones functioneren niet meer correct.
- API-responses bevatten ontbrekende velden.
- De gegevens staan vaak nog wel in de database.

Oorzaak

Dit gebeurt meestal wanneer iemand:

1. Het Production Admin Panel opent.
2. De **Content-Type Builder** gebruikt.
3. Een nieuw veld toevoegt.
4. De gegenereerde schemabestanden niet commit naar Git.
5. Coolify vervolgens een rebuild uitvoert vanaf GitHub.

Resultaat:

Database = Nieuw schema

Git Repository = Oud schema

Na de deployment:

Strapi gebruikt het oude schema.

Checklist voor herstel

Stap 1 Controleer de database.

Verifieer dat de content nog aanwezig is.

Stap 2 Controleer de huidige Git-repository.

Bevestig welke schemabestanden ontbreken.

Stap 3 Controleer de draaiende container.

Herstel de volgende mappen:

src/api

src/components

Stap 4 Commit de herstellende bestanden naar de Git-repository.

Stap 5 Voer een nieuwe deployment uit.

Stap 6 Controleer of de Content Manager weer volledig functioneert.

12. Woordenlijst

Term	Betekenis
Headless CMS	Een CMS dat content opslaat en levert via API, zonder ingebouwde front-end weergavelaag
SSG	Statische sitegeneratie — pagina's worden gebouwd naar gewone HTML-bestanden tijdens het bouwen, niet bij elk verzoek
Strapi	Open-source Node.js headless CMS gebruikt als backend in dit project
SvelteKit	Full-stack webframework gebouwd op Svelte; hier gebruikt in SSG-modus
Collection type	Een Strapi-inhoudstype dat meerdere invoeren toestaat (zoals een databasetabel)
Single type	Een Strapi-inhoudstype met precies één invoer (bijv. Navigatie)
Component	Een herbruikbare groep velden in Strapi, ingebed in een inhoudstype
Slug	Een URL-veilige versie van een titel, bijv. "Mijn Project" → "mijn-project"
UID field	Een Strapi-veldtype dat automatisch een unieke slug genereert van een ander veld
API token	Een geheime tekenreeks gebruikt voor het authenticeren van verzoeken aan de Strapi REST API
adapter-static	Een SvelteKit-adaptor die een map met statische HTML/CSS/JS-bestanden produceert
Coolify	Een zelf-gehoste PaaS (zoals Heroku) gebruikt voor het deployen van beide projecten
Deploy webhook	Een HTTP-aanroep die Coolify triggert om een nieuwe build en deployment te starten
entries()	Een SvelteKit-functie die alle vooraf renderbare paden voor een dynamische route opsomt
Runes	Svelte 5's nieuwe reactiviteitsprimitieven: \$props(), \$state(), \$derived(), enz.

13. Huidige status

Huidig process

De basis opzet van het project is opgezet en er is een begin gemaakt om alle data binnen te halen

Overgebleven taken

- P5 op een goede manier inladen het liefste tijdens het builden
- Styling toepassen
- Alle datavelden toevoegen aan Strapi
- Alle data ophalen
- Content toevoegen
- Controleer lighthouse

Einde van de documentatie — responsibleIT Headless CMS

